Principles of Computer Science II

Data Structure and OOP

Marco Zecchini

Sapienza University of Rome

Lecture 4

Counting Frequencies Problem

Problem: Given a list of numbers, count how many times each number appears.

Example:

$$[2, 3, 2, 5, 3, 2, 7, 5, 3] \longrightarrow \begin{cases} 2:3 \\ 3:3 \\ 5:2 \\ 7:1 \end{cases}$$

We will solve this problem using two different data structures:

- A **list** of pairs (number, count)
- A **dictionary**

Solution 1: Using a List

Idea: Maintain a list of pairs `[number, count]`. For each number in the input list, search if it already exists; if yes, increase its count, otherwise append it.

```
counts = []
for n in numbers:
    found = False
    for pair in counts:
        if pair[0] == n:
            pair[1] += 1
            found = True
            break
    if not found:
        counts.append([n, 1])
```

Solution 2: Using a Dictionary

Idea: Use a dictionary that directly associates each number to its count (remember: in a dictionary we access the value associated with a key in constant time).

```
counts = {}
for n in numbers:
    if n in counts:
        counts[n] += 1
    else:
        counts[n] = 1
```

Question

Which is better of the two in terms of time complexity? And why?

Summary and Discussion

Same problem, different data structures:

Approach	**Data Structure**	**Time Complexity**
Naive counting	List of pairs	$O(n^2)$
Efficient counting	Dictionary	$O(n)$

Key takeaway: Choosing the right **data structure** can drastically improve algorithm performance, even when solving the *same* problem.

Outline of the lecture

- Until now, most of the times, we have always worked with primitive data types (integer, float, string, char)...

Outline of the lecture

- Until now, most of the times, we have always worked with primitive data types (integer, float, string, char)...
- ...and combined them into two simple data structure: which are these?

Outline of the lecture

- Until now, most of the times, we have always worked with primitive data types (integer, float, string, char)...
- ...and combined them into two simple data structure: which are these?
- List and Dictionaries

Outline of the lecture

- Until now, most of the times, we have always worked with primitive data types (integer, float, string, char)...
- ...and combined them into two simple data structure: which are these?
- List and Dictionaries
- In the rest of the lecture (and partially of the course), we will see more complex data structure

Outline of the lecture

- Until now, most of the times, we have always worked with primitive data types (integer, float, string, char)...
- ...and combined them into two simple data structure: which are these?
- List and Dictionaries
- In the rest of the lecture (and partially of the course), we will see more complex data structure
- ... and how to build them in Python (or any other programming language)

A question about code you already write

```
numbers = [2, 3, 2]
numbers.append(5)

counts = {2: 3, 5: 2}
print(counts.keys())

dna = "acgt"
print(dna.upper())
```

- append, keys, upper look like functions... but we call them **with a dot, after a variable**. Why?

A question about code you already write

```
numbers = [2, 3, 2]
numbers.append(5)

counts = {2: 3, 5: 2}
print(counts.keys())

dna = "acgt"
print(dna.upper())
```

- append, keys, upper look like functions... but we call them **with a dot, after a variable**. Why?
- Because numbers, counts, dna are not just values: each one carries its **data** *and* the **functions** that operate on it

A question about code you already write

```
numbers = [2, 3, 2]
numbers.append(5)

counts = {2: 3, 5: 2}
print(counts.keys())

dna = "acgt"
print(dna.upper())
```

- append, keys, upper look like functions... but we call them **with a dot, after a variable**. Why?
- Because numbers, counts, dna are not just values: each one carries its **data** *and* the **functions** that operate on it
- Such a "data + functions" thing is called an **object**; its functions are called **methods**, and `object.method()` asks the object to do something

A question about code you already write

```
numbers = [2, 3, 2]
numbers.append(5)

counts = {2: 3, 5: 2}
print(counts.keys())

dna = "acgt"
print(dna.upper())
```

- `append`, `keys`, `upper` look like functions... but we call them **with a dot, after a variable**. Why?
- Because `numbers`, `counts`, `dna` are not just values: each one carries its **data** *and* the **functions** that operate on it
- Such a "data + functions" thing is called an **object**; its functions are called **methods**, and `object.method()` asks the object to do something
- `list`, `dict` and `str` are the **classes** these objects are built from. Let's see what this means

Class vs Object

Class

A **class** is a **template** for creating objects. It defines the *data* (attributes) and the *behaviours* (methods) that every object of that class will have.

Object

An **object** (or *instance*) is a concrete thing built from the template, with its **own** data.

Quick Intuition

Think of a class as a **recipe**. An **object** is a dish prepared from that recipe: one recipe, many dishes.

Anatomy of a class

```
class DNASequence:

    def __init__(self, name, seq):
        self.name = name
        self.seq = seq

    def length(self):
        return len(self.seq)

    def gc_content(self):
        g = self.seq.count("G")
        c = self.seq.count("C")
        return (g + c) / self.length()
```

Anatomy of a class

```
class DNASequence:

    def __init__(self, name, seq):
        self.name = name
        self.seq = seq

    def length(self):
        return len(self.seq)

    def gc_content(self):
        g = self.seq.count("G")
        c = self.seq.count("C")
        return (g + c) / self.length()
```

- `class DNASequence:` gives the template a **name**

Anatomy of a class

```
class DNASequence:

    def __init__(self, name, seq):
        self.name = name
        self.seq = seq

    def length(self):
        return len(self.seq)

    def gc_content(self):
        g = self.seq.count("G")
        c = self.seq.count("C")
        return (g + c) / self.length()
```

- `class DNASequence:` gives the template a **name**
- `__init__` is the **constructor**: it runs when a new object is created

Anatomy of a class

```
class DNASequence:

    def __init__(self, name, seq):
        self.name = name
        self.seq = seq

    def length(self):
        return len(self.seq)

    def gc_content(self):
        g = self.seq.count("G")
        c = self.seq.count("C")
        return (g + c) / self.length()
```

- `class DNASequence:` gives the template a **name**
- `__init__` is the **constructor**: it runs when a new object is created
- `self.name`, `self.seq` are **attributes**: the data stored in each object

Anatomy of a class

```
class DNASequence:

    def __init__(self, name, seq):
        self.name = name
        self.seq = seq

    def length(self):
        return len(self.seq)

    def gc_content(self):
        g = self.seq.count("G")
        c = self.seq.count("C")
        return (g + c) / self.length()
```

- `class DNASequence:` gives the template a **name**
- `__init__` is the **constructor**: it runs when a new object is created
- `self.name`, `self.seq` are **attributes**: the data stored in each object
- `length`, `gc_content` are **methods**: functions that work on the object's data

Anatomy of a class

```
class DNASequence:

    def __init__(self, name, seq):
        self.name = name
        self.seq = seq

    def length(self):
        return len(self.seq)

    def gc_content(self):
        g = self.seq.count("G")
        c = self.seq.count("C")
        return (g + c) / self.length()
```

- `class DNASequence:` gives the template a **name**
- `__init__` is the **constructor**: it runs when a new object is created
- `self.name`, `self.seq` are **attributes**: the data stored in each object
- `length`, `gc_content` are **methods**: functions that work on the object's data
- `self` is **the object itself**: always the first parameter of a method

Creating and using objects

```
s1 = DNASequence("gene_A", "ATGCGC")
s2 = DNASequence("gene_B", "ATATAT")

print(s1.name, s1.length())       # gene_A 6
print(s2.name, s2.gc_content())   # gene_B 0.0

s2.seq = s2.seq + "GG"            # change the state of s2 only
print(s2.gc_content())            # 0.25
print(s1.gc_content())            # 0.666...  s1 is untouched
```

Creating and using objects

```
s1 = DNASequence("gene_A", "ATGCGC")
s2 = DNASequence("gene_B", "ATATAT")

print(s1.name, s1.length())       # gene_A 6
print(s2.name, s2.gc_content())   # gene_B 0.0

s2.seq = s2.seq + "GG"            # change the state of s2 only
print(s2.gc_content())            # 0.25
print(s1.gc_content())            # 0.666...  s1 is untouched
```

- `DNASequence(...)` **creates** a new object and calls `__init__` on it

Creating and using objects

```
s1 = DNASequence("gene_A", "ATGCGC")
s2 = DNASequence("gene_B", "ATATAT")

print(s1.name, s1.length())       # gene_A 6
print(s2.name, s2.gc_content())   # gene_B 0.0

s2.seq = s2.seq + "GG"            # change the state of s2 only
print(s2.gc_content())            # 0.25
print(s1.gc_content())            # 0.666...  s1 is untouched
```

- `DNASequence(...)` **creates** a new object and calls `__init__` on it
- Each object has its **own** attributes: changing s2 does not affect s1

Creating and using objects

```
s1 = DNASequence("gene_A", "ATGCGC")
s2 = DNASequence("gene_B", "ATATAT")

print(s1.name, s1.length())       # gene_A 6
print(s2.name, s2.gc_content())   # gene_B 0.0

s2.seq = s2.seq + "GG"            # change the state of s2 only
print(s2.gc_content())            # 0.25
print(s1.gc_content())            # 0.666...  s1 is untouched
```

- `DNASequence(...)` **creates** a new object and calls `__init__` on it
- Each object has its **own** attributes: changing `s2` does not affect `s1`
- Inside a method, `self` is the object the method was called on: in `s1.length()` we have `self = s1`

Two kinds of methods

Methods that **ask** something

They read the attributes and return an answer; the object stays the same.

```
def length(self):
    return len(self.seq)
```

Methods that **change** the object

They modify the attributes; afterwards the object has a new *state*.

```
def append(self, bases):
    self.seq = self.seq + bases
```

Keep this in mind

Every data structure we will see has both kinds: `is_empty()` and `peek()` ask, `push()` and `pop()` change.

An attribute can hold another object (or nothing)

Attributes are not limited to numbers and strings: they can refer to **other objects**, or to `None` ("nothing").

```
class Node:
    def __init__(self, value):
        self.value = value    # the data
        self.next = None      # a reference to another Node (or None)

a = Node(2)
b = Node(5)
a.next = b            # now a "points to" b
print(a.next.value)   # 5
```

An attribute can hold another object (or nothing)

Attributes are not limited to numbers and strings: they can refer to **other objects**, or to `None` ("nothing").

```
class Node:
    def __init__(self, value):
        self.value = value    # the data
        self.next = None      # a reference to another Node (or None)

a = Node(2)
b = Node(5)
a.next = b            # now a "points to" b
print(a.next.value)   # 5
```

- `a.next = b` does not *copy* `b`: it stores a **reference** to the same object

An attribute can hold another object (or nothing)

Attributes are not limited to numbers and strings: they can refer to **other objects**, or to `None` ("nothing").

```
class Node:
    def __init__(self, value):
        self.value = value    # the data
        self.next = None      # a reference to another Node (or None)

a = Node(2)
b = Node(5)
a.next = b            # now a "points to" b
print(a.next.value)   # 5
```

- `a.next = b` does not *copy* `b`: it stores a **reference** to the same object
- Chaining objects this way is exactly how we will build a **linked list**

Why classes for data structures?

Interface vs implementation

A class separates **what** you can do (the methods: the *interface*) from **how** it is done inside (the attributes: the *implementation*).

Why classes for data structures?

Interface vs implementation

A class separates **what** you can do (the methods: the *interface*) from **how** it is done inside (the attributes: the *implementation*).

- Whoever **uses** `DNASequence` only needs `length()` and `gc_content()`
- Whoever **writes** it decides how the sequence is stored (a string? a list?) and can change it later: `s1.gc_content()` still works

Why classes for data structures?

Interface vs implementation

A class separates **what** you can do (the methods: the *interface*) from **how** it is done inside (the attributes: the *implementation*).

- Whoever **uses** `DNASequence` only needs `length()` and `gc_content()`
- Whoever **writes** it decides how the sequence is stored (a string? a list?) and can change it later: `s1.gc_content()` still works
- You rely on this every day: `append` and `pop` on a `list`, without knowing how it is stored in memory

Why classes for data structures?

Interface vs implementation

A class separates **what** you can do (the methods: the *interface*) from **how** it is done inside (the attributes: the *implementation*).

- Whoever **uses** `DNASequence` only needs `length()` and `gc_content()`
- Whoever **writes** it decides how the sequence is stored (a string? a list?) and can change it later: `s1.gc_content()` still works
- You rely on this every day: `append` and `pop` on a `list`, without knowing how it is stored in memory
- Today we will see this idea **in action**: each data structure is a class with a small, clear interface

Linked List

Idea: A sequence of nodes, each containing a value and a reference to the next node.

```
class Node:
    def __init__(self, value):
        self.value = value    # the data
        self.next = None      # reference to the next Node
```

Each node is an **object** of class `Node`; the list is just the chain of `next` references starting from the first node (`head`).

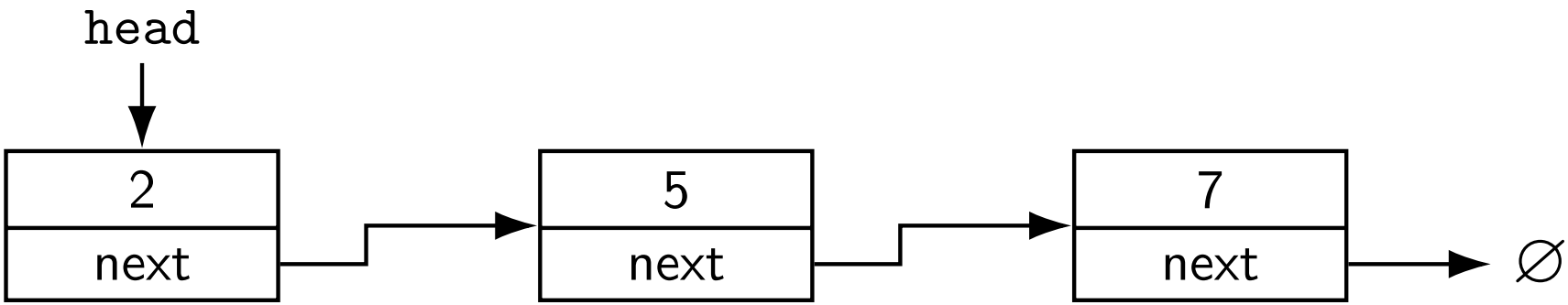
Advantages:

- Efficient insertions/deletions at both ends
- Dynamic memory usage

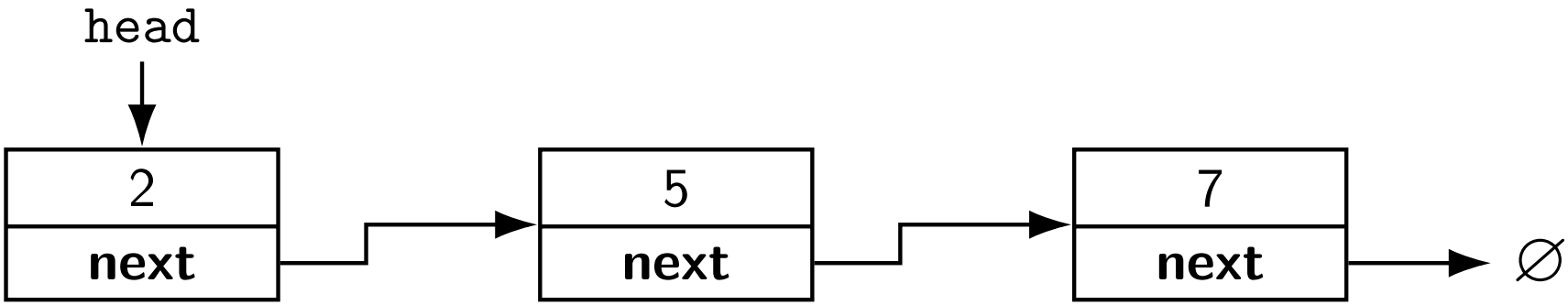
Drawbacks:

- Random access not possible ($O(n)$)
- Extra memory for pointers

Singly Linked List — Structure

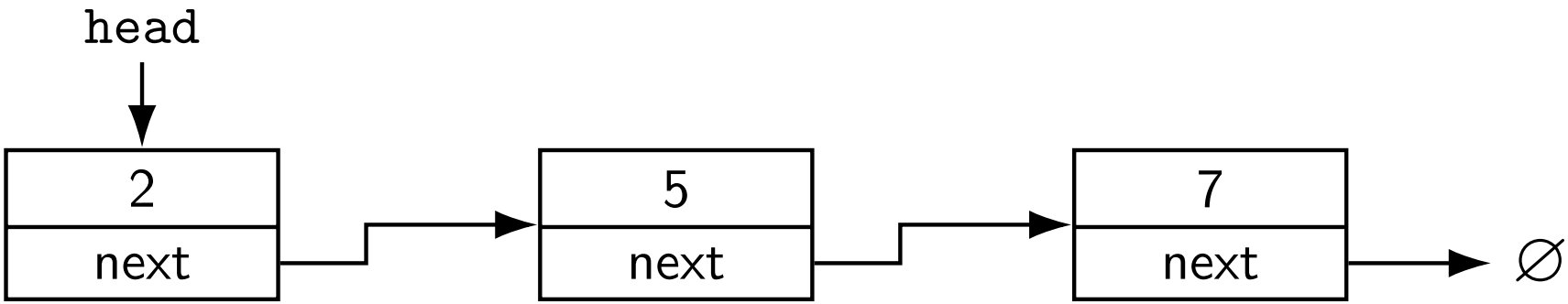


Singly Linked List — Structure



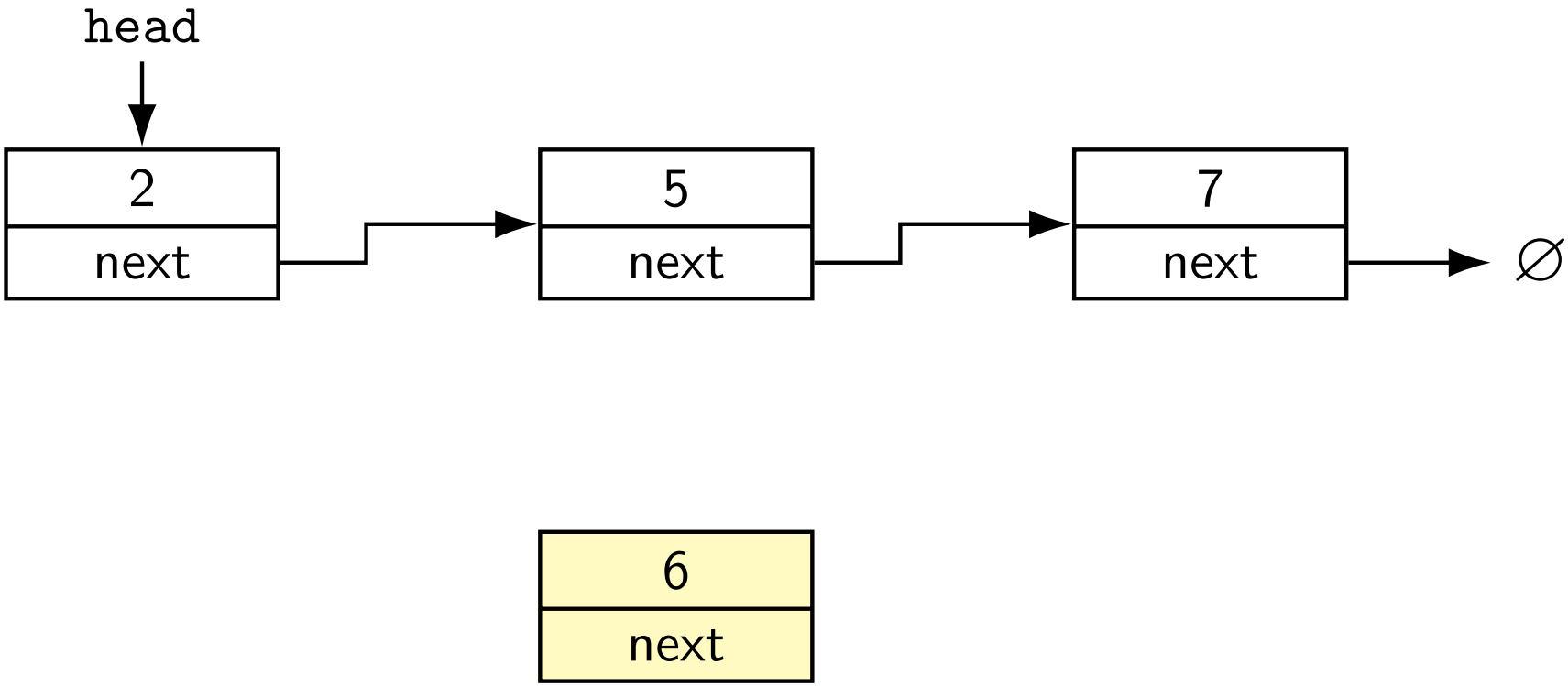
Each node stores **data** and a pointer to **next**

Linked List — Insertion (between two nodes)



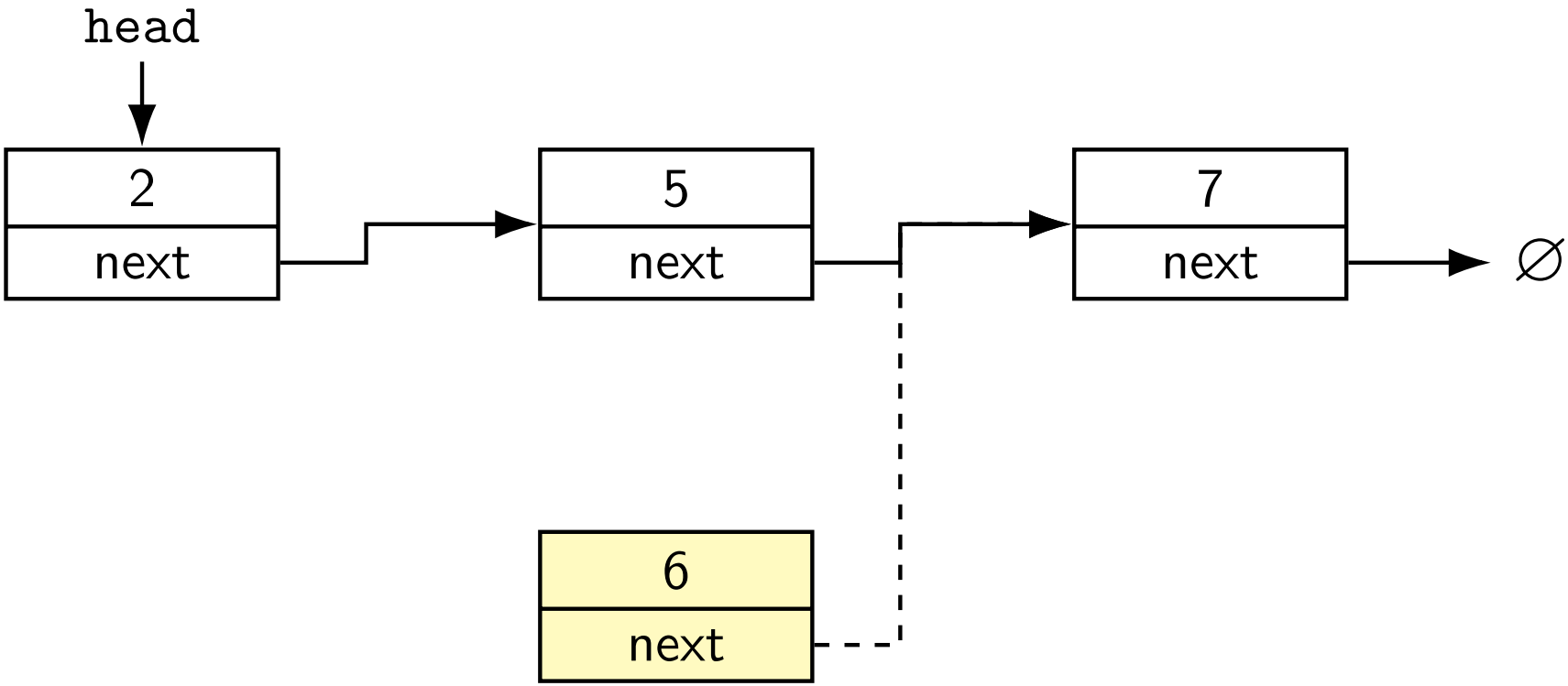
Goal: insert **6** after node **5**

Linked List — Insertion (between two nodes)



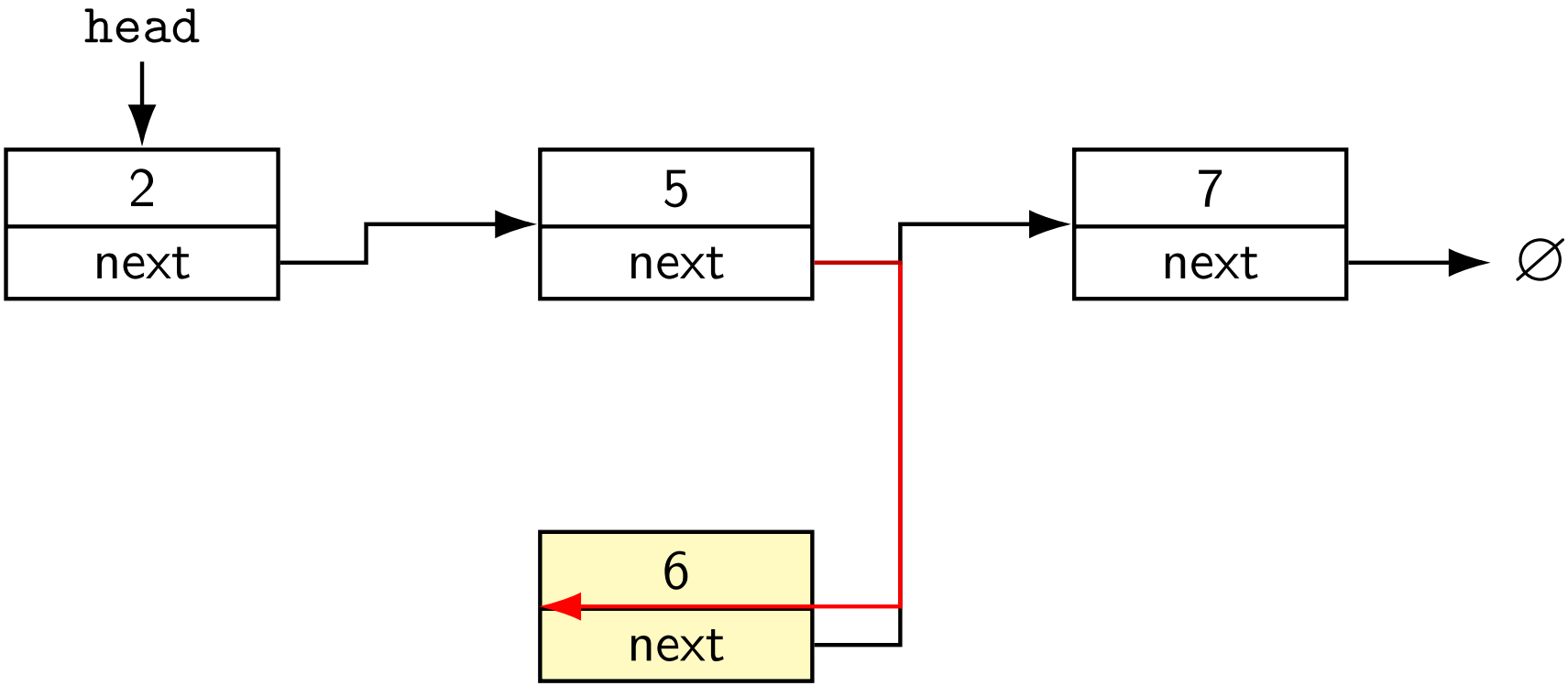
1) Allocate new node **X=6**

Linked List — Insertion (between two nodes)



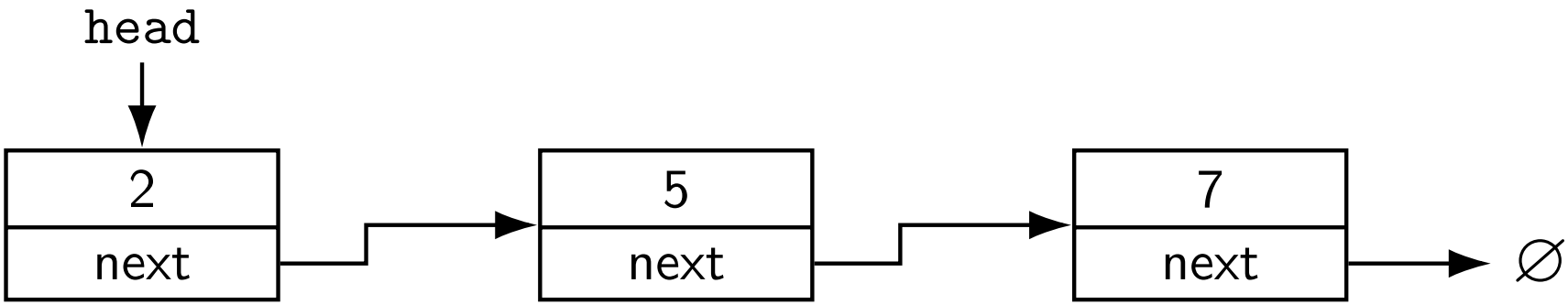
2) Set `X.next = 7`

Linked List — Insertion (between two nodes)



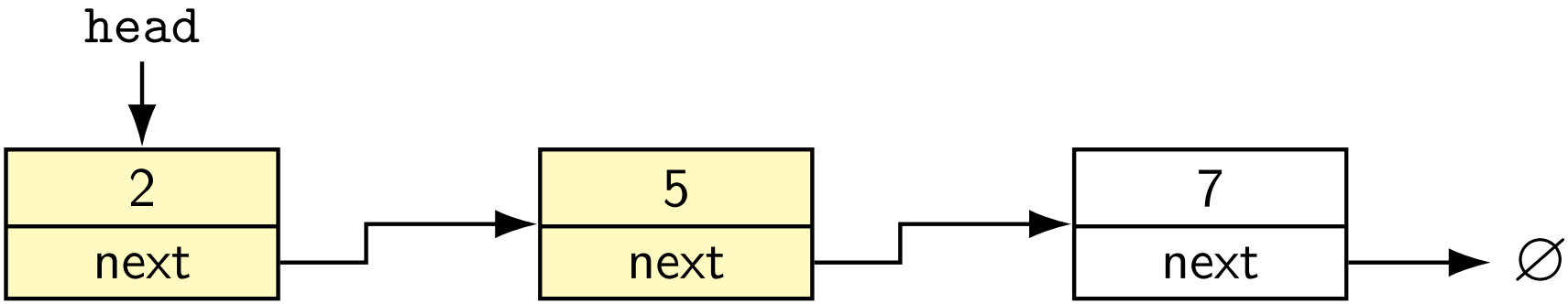
3) Set `5.next = X` $\rightarrow$ insertion complete

Linked List — Deletion (bypass the target node)



Goal: delete node **5** (have pointer to **prev**=2)

Linked List — Deletion (bypass the target node)



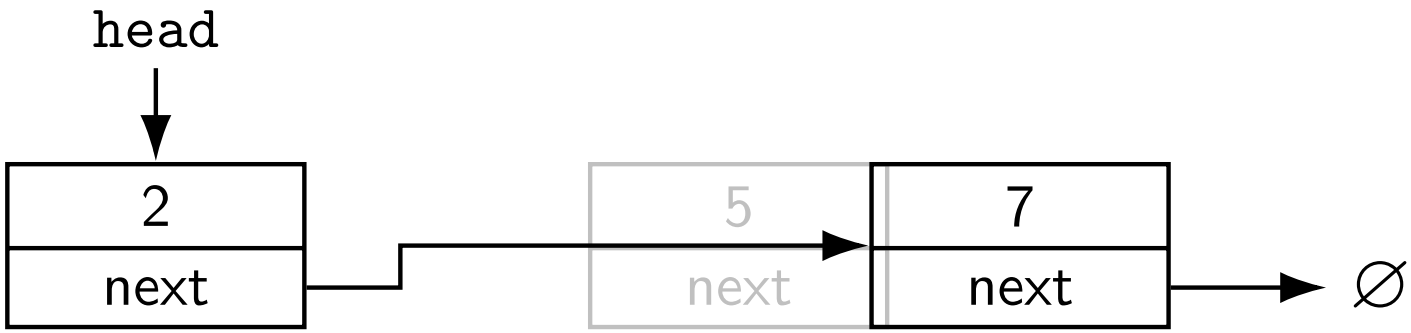
1) Identify **prev** (2) and **target** (5)

Linked List — Deletion (bypass the target node)



2) Set `prev.next = target.next`

Linked List — Deletion (bypass the target node)



3) (Optional) deallocate **target**

Linked List in Python

Two classes working together

```
class Node:
  def __init__(self, value):
    self.value = value
    self.next = None

class LinkedList:
  def __init__(self):
    self.head = None            # empty list

  def push_front(self, value):
    new = Node(value)
    new.next = self.head
    self.head = new

  def insert_after(self, prev, value):
    new = Node(value)           # 1) allocate the new node
    new.next = prev.next        # 2) new points to prev's successor
    prev.next = new             # 3) prev points to new

  def delete_after(self, prev):
    target = prev.next
    prev.next = target.next     # bypass the target node
```

Linked List in Python (cont.)

Using it: the example of the pictures

```
ll = LinkedList()
ll.push_front(7)
ll.push_front(5)
ll.push_front(2)            # head -> 2 -> 5 -> 7

n5 = ll.head.next           # the Node object containing 5
ll.insert_after(n5, 6)      # head -> 2 -> 5 -> 6 -> 7
ll.delete_after(ll.head)    # head -> 2 -> 6 -> 7

node = ll.head              # walk the chain of references
while node is not None:
    print(node.value)       # 2  6  7
    node = node.next
```

Linked List in Python (cont.)

Using it: the example of the pictures

```
ll = LinkedList()
ll.push_front(7)
ll.push_front(5)
ll.push_front(2)            # head -> 2 -> 5 -> 7

n5 = ll.head.next           # the Node object containing 5
ll.insert_after(n5, 6)      # head -> 2 -> 5 -> 6 -> 7
ll.delete_after(ll.head)    # head -> 2 -> 6 -> 7

node = ll.head              # walk the chain of references
while node is not None:
    print(node.value)       # 2  6  7
    node = node.next
```

- The `LinkedList` object stores only `head`; the data lives in the `Node` objects
- `insert_after` **changes** the state of the list: its three lines are the three steps of the picture

Do we need Linked List in Python?

Question

In a framework like Python, do we actually need them?

Do we need Linked List in Python?

Question

In a framework like Python, do we actually need them?

No, we actually don't! We already have Lists that have the same benefits: we essentially saw how Lists are implemented behind the hood (or a way to do that)

Stack and Queue

Two fundamental ways to organize and manage elements:

Stack — LIFO (Last In, First Out)

- The last element added is the first one to be removed.
- Think of a stack of plates: you remove the top one first.
- Useful when an algorithm needs to "go back": recursion, undo mechanisms, backtracking.

Queue — FIFO (First In, First Out)

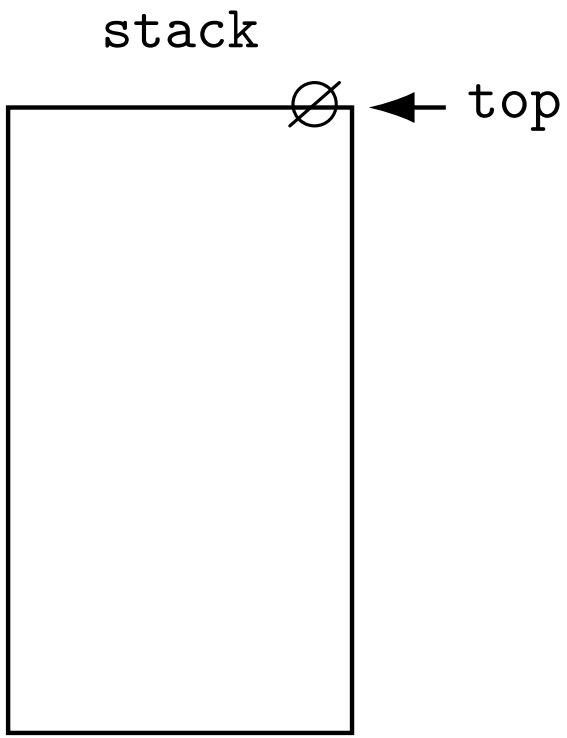
- The first element added is the first one to be removed.
- Like a line at the supermarket: first come, first served.
- Used when order must be respected: task scheduling, BFS, event handling.

Stack and Queue (cont.)

Why are they important?

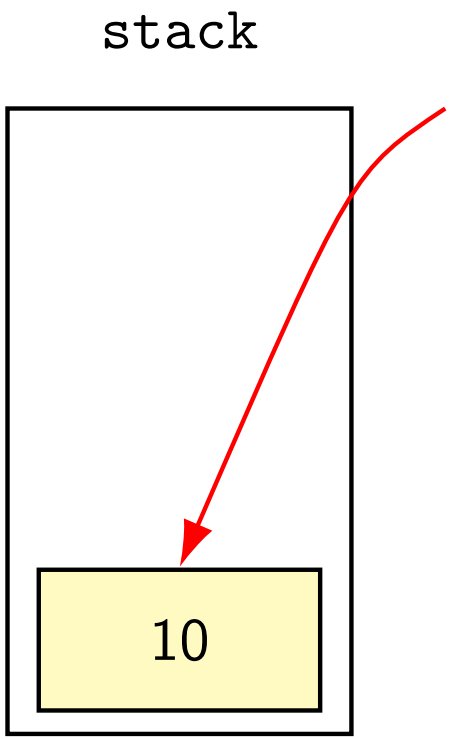
- They impose a simple but powerful order on how elements are processed.
- They help control the flow of algorithms clearly and predictably.
- They mirror natural behaviors (stacks, lines) $\rightarrow$ easy to understand, essential in computing.

Stack (LIFO) — Structure and Push



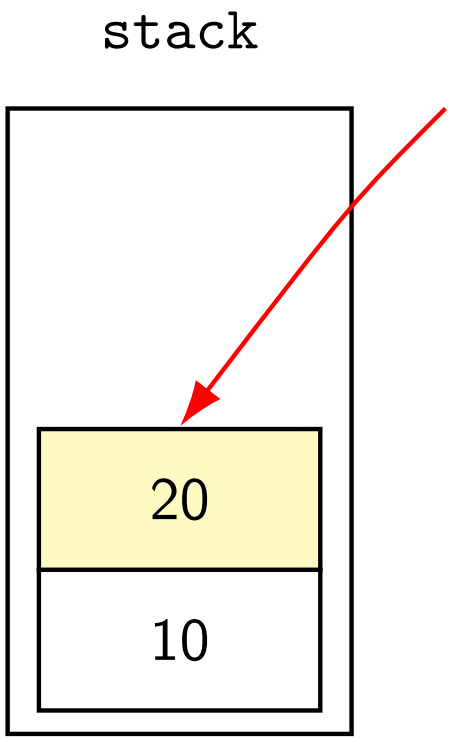
Empty stack: `top = null`

Stack (LIFO) — Structure and Push



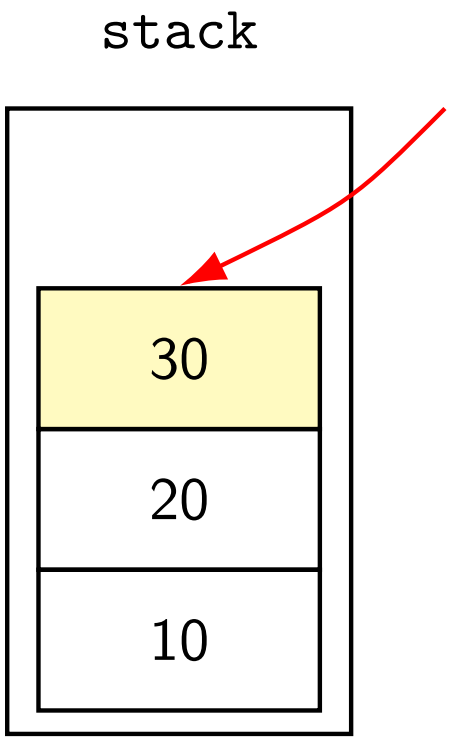
push(10): place at bottom, `top` $\rightarrow$ 10

Stack (LIFO) — Structure and Push



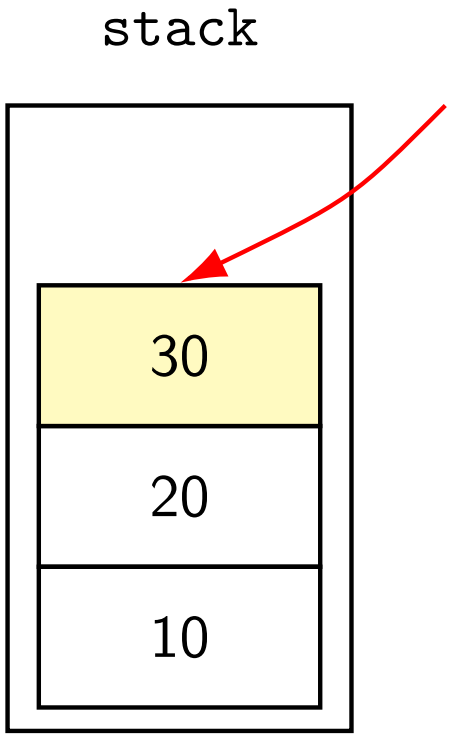
push(20): new item goes on top

Stack (LIFO) — Structure and Push



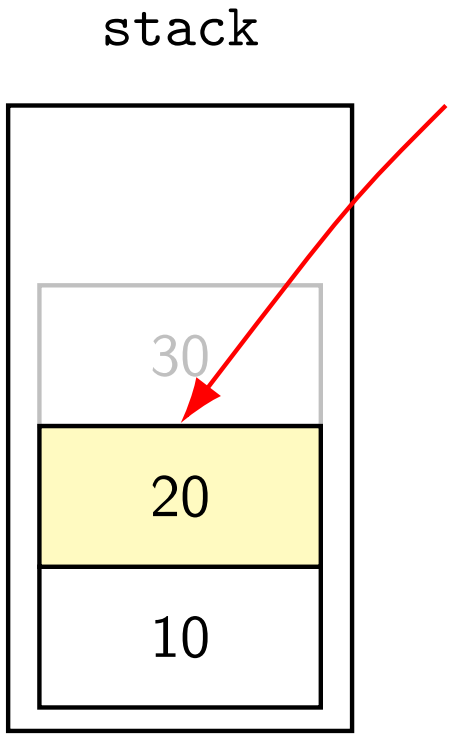
push(30): LIFO grows upwards

Stack (LIFO) — Pop (Deletion)



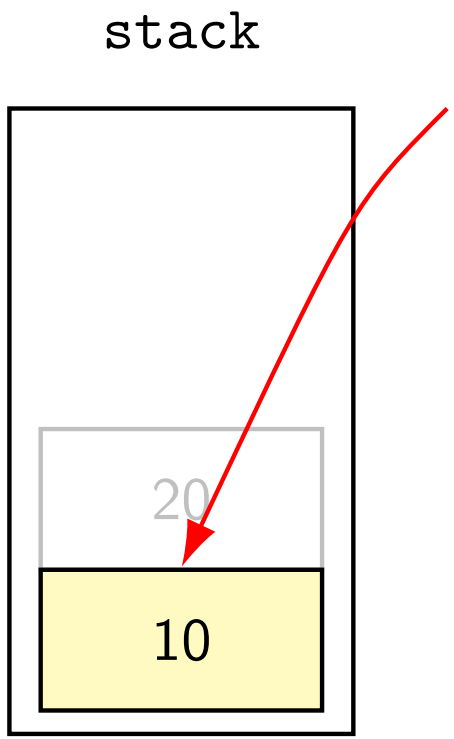
pop() removes the **top** element (30)

Stack (LIFO) — Pop (Deletion)



After pop: `top` $\rightarrow$ 20

Stack (LIFO) — Pop (Deletion)



pop() again: `top` $\rightarrow$ 10

Stack in Python

Minimal Stack implementation

```
class Stack:
  def __init__(self):
    self.items = []
  def is_empty(self):
    return len(self.items) == 0
  def push(self, item):
    self.items.append(item)
  def pop(self):
    if not self.is_empty():
      return self.items.pop()
    raise IndexError("Stack is empty.")
  def peek(self):
    if not self.is_empty():
      return self.items[-1]
    raise IndexError("Stack is empty.")

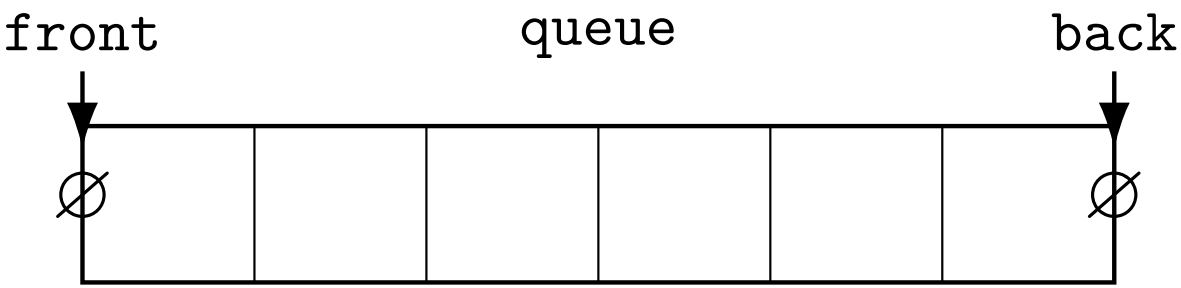
my_stack = Stack()
my_stack.push(10)
my_stack.push(20)
my_stack.push(30)
print(my_stack.pop())  # 30
print(my_stack.peek()) # 20
```

Same interface, different implementation

A Stack made of linked Nodes

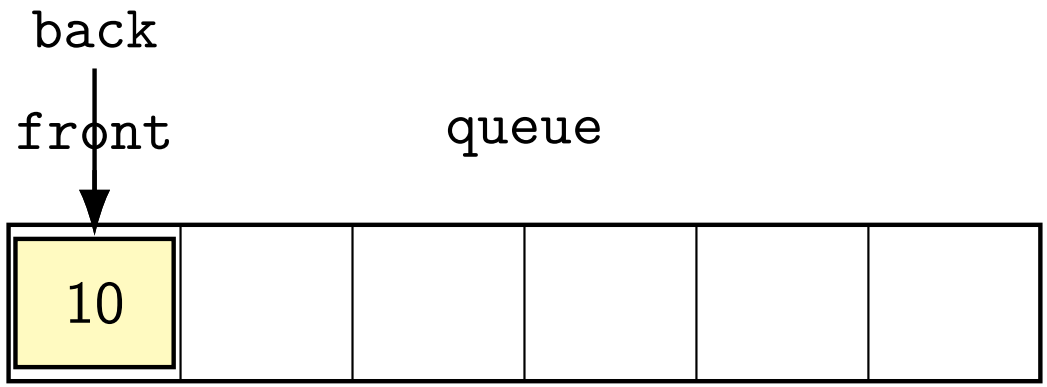
```
class LinkedStack:
  def __init__(self):
    self.top = None                 # no list: just a reference to the top Node
  def is_empty(self):
    return self.top is None
  def push(self, item):
    new = Node(item)
    new.next = self.top
    self.top = new
  def pop(self):
    if self.is_empty():
      raise IndexError("Stack is empty.")
    item = self.top.value
    self.top = self.top.next
    return item
  def peek(self):
    if self.is_empty():
      raise IndexError("Stack is empty.")
    return self.top.value
my_stack = LinkedStack()  # the code that uses it does not change!
my_stack.push(10)
my_stack.push(20)
print(my_stack.pop())     # 20
```

Queue (FIFO) — Structure and Enqueue



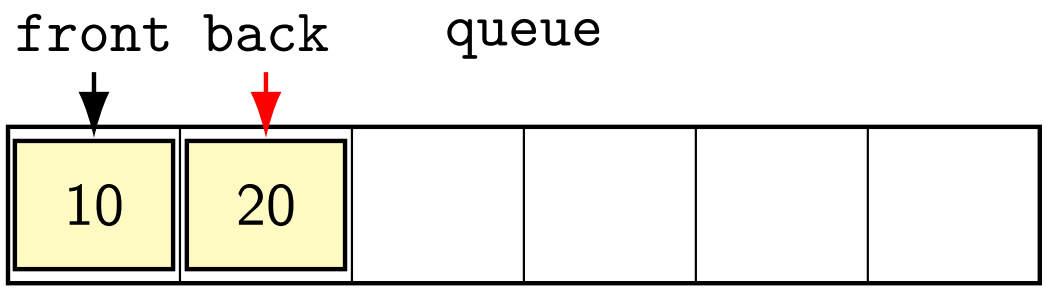
Empty queue: `front = back = null`

Queue (FIFO) — Structure and Enqueue



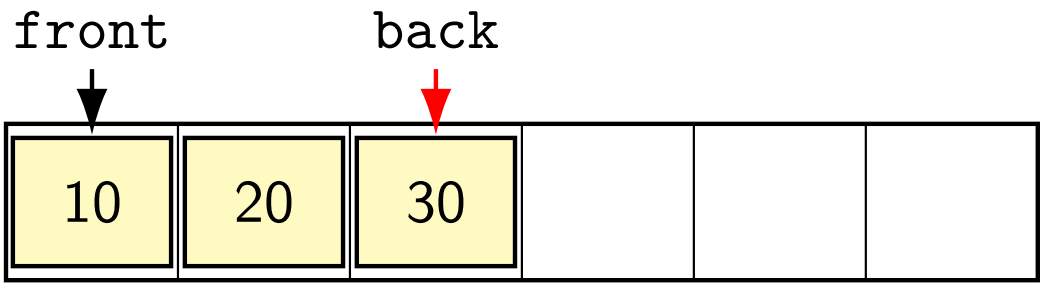
enqueue(10): first element sets both `front` and `back`

Queue (FIFO) — Structure and Enqueue



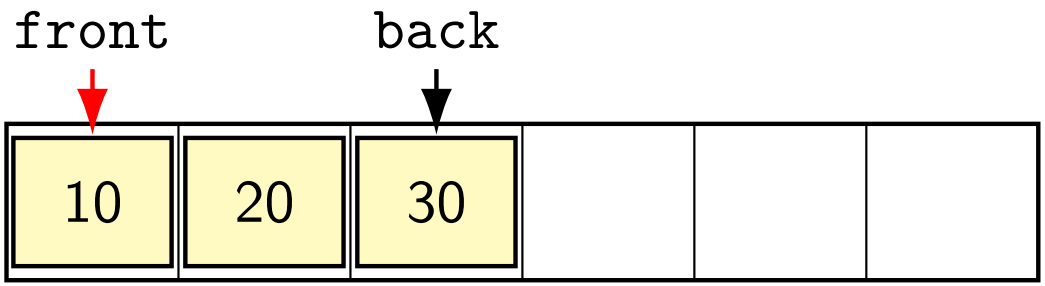
enqueue(20): insert at **back**

Queue (FIFO) — Structure and Enqueue



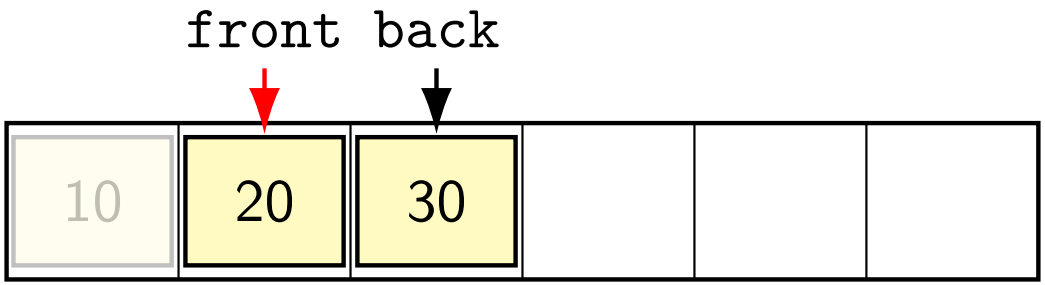
enqueue(30): FIFO grows to the right

Queue (FIFO) — Dequeue (Deletion)



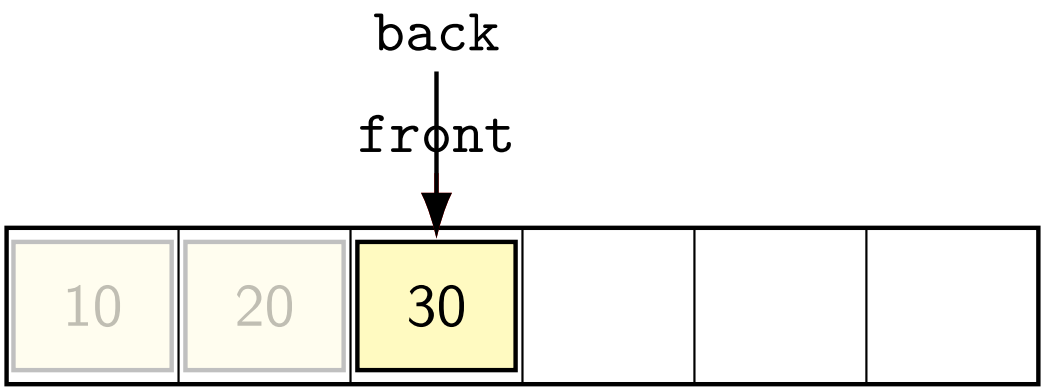
dequeue() removes the **front** element (10)

Queue (FIFO) — Dequeue (Deletion)



After dequeue: `front` $\rightarrow$ 20

Queue (FIFO) — Dequeue (Deletion)



dequeue() again: `front` $\rightarrow$ 30

Queue in Python

A Queue made of linked Nodes

```
class Queue:
  def __init__(self):
    self.front = None    # first Node: we dequeue from here
    self.back = None     # last Node: we enqueue here
  def is_empty(self):
    return self.front is None
  def enqueue(self, item):
    new = Node(item)
    if self.is_empty(): self.front = new
    else: self.back.next = new
    self.back = new
  def dequeue(self):
    if self.is_empty():
      raise IndexError("Queue is empty.")
    item = self.front.value
    self.front = self.front.next
    if self.front is None: self.back = None
    return item
  # peek(): as in LinkedStack, but returns self.front.value
my_queue = Queue()
my_queue.enqueue("A")
my_queue.enqueue("B")
print(my_queue.dequeue())  # A
```

Queues in practice: `collections.deque`

```
from collections import deque
q = deque()
q.append("A")        # enqueue
q.append("B")
print(q.popleft())  # dequeue -> A, in O(1)
```

Queues in practice: `collections.deque`

```
from collections import deque
q = deque()
q.append("A")        # enqueue
q.append("B")
print(q.popleft())   # dequeue -> A, in O(1)
```

- deque (*double-ended queue*) is Python's ready-made queue: add and remove at **both ends** in $O(1)$. In real code, use it

Queues in practice: `collections.deque`

```
from collections import deque
q = deque()
q.append("A")        # enqueue
q.append("B")
print(q.popleft())   # dequeue -> A, in O(1)
```

- deque (*double-ended queue*) is Python's ready-made queue: add and remove at **both ends** in $O(1)$. In real code, use it
- Why not a plain `list`? Removing the **first** element with `pop(0)` forces all the others to shift: $O(n)$ per dequeue

Queues in practice: `collections.deque`

```
from collections import deque
q = deque()
q.append("A")        # enqueue
q.append("B")
print(q.popleft())   # dequeue -> A, in O(1)
```

- deque (*double-ended queue*) is Python's ready-made queue: add and remove at **both ends** in $O(1)$. In real code, use it
- Why not a plain `list`? Removing the **first** element with `pop(0)` forces all the others to shift: $O(n)$ per dequeue
- Our `Queue` class shows what happens **inside**: `append` and `popleft` are our `enqueue` and `dequeue` on the chain of `Nodes`

Stack and Queue through the OOP lens

- Both classes have the **same shape**: a couple of attributes plus a small **interface** of four methods

Stack and Queue through the OOP lens

- Both classes have the **same shape**: a couple of attributes plus a small **interface** of four methods
- `is_empty` and `peek` **ask**; `push`/`pop` and `enqueue`/`dequeue` **change** the state

Stack and Queue through the OOP lens

- Both classes have the **same shape**: a couple of attributes plus a small **interface** of four methods
- `is_empty` and `peek` **ask**; `push`/`pop` and `enqueue`/`dequeue` **change** the state
- `Stack` stores the items in a `list`; `LinkedStack` and `Queue` in a chain of `Nodes`: the **implementation** differs, the **interface** does not

Stack and Queue through the OOP lens

- Both classes have the **same shape**: a couple of attributes plus a small **interface** of four methods
- `is_empty` and `peek` **ask**; `push`/`pop` and `enqueue`/`dequeue` **change** the state
- `Stack` stores the items in a `list`; `LinkedStack` and `Queue` in a chain of `Nodes`: the **implementation** differs, the **interface** does not
- The user of the class never touches `items`, `top` or `front` directly: only the methods do

Tree Data Structures

What is a Tree?

A **tree** is a hierarchical data structure composed of **nodes** connected by **edges**. It represents relationships like those found in family trees, organization charts, or file systems.



Tree Data Structures

What is a Tree?

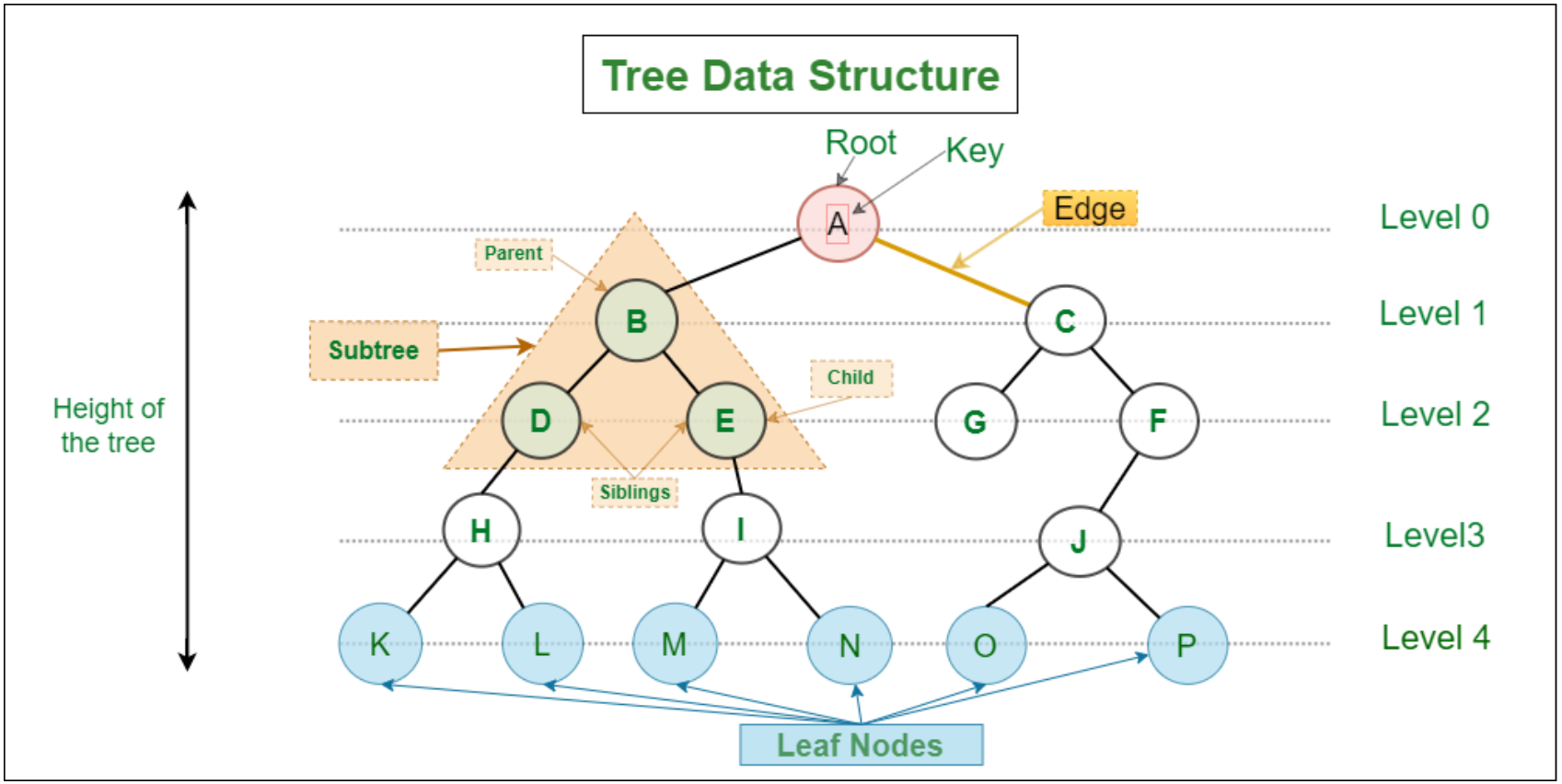
A **tree** is a hierarchical data structure composed of **nodes** connected by **edges**. It represents relationships like those found in family trees, organization charts, or file systems.

Why Trees?

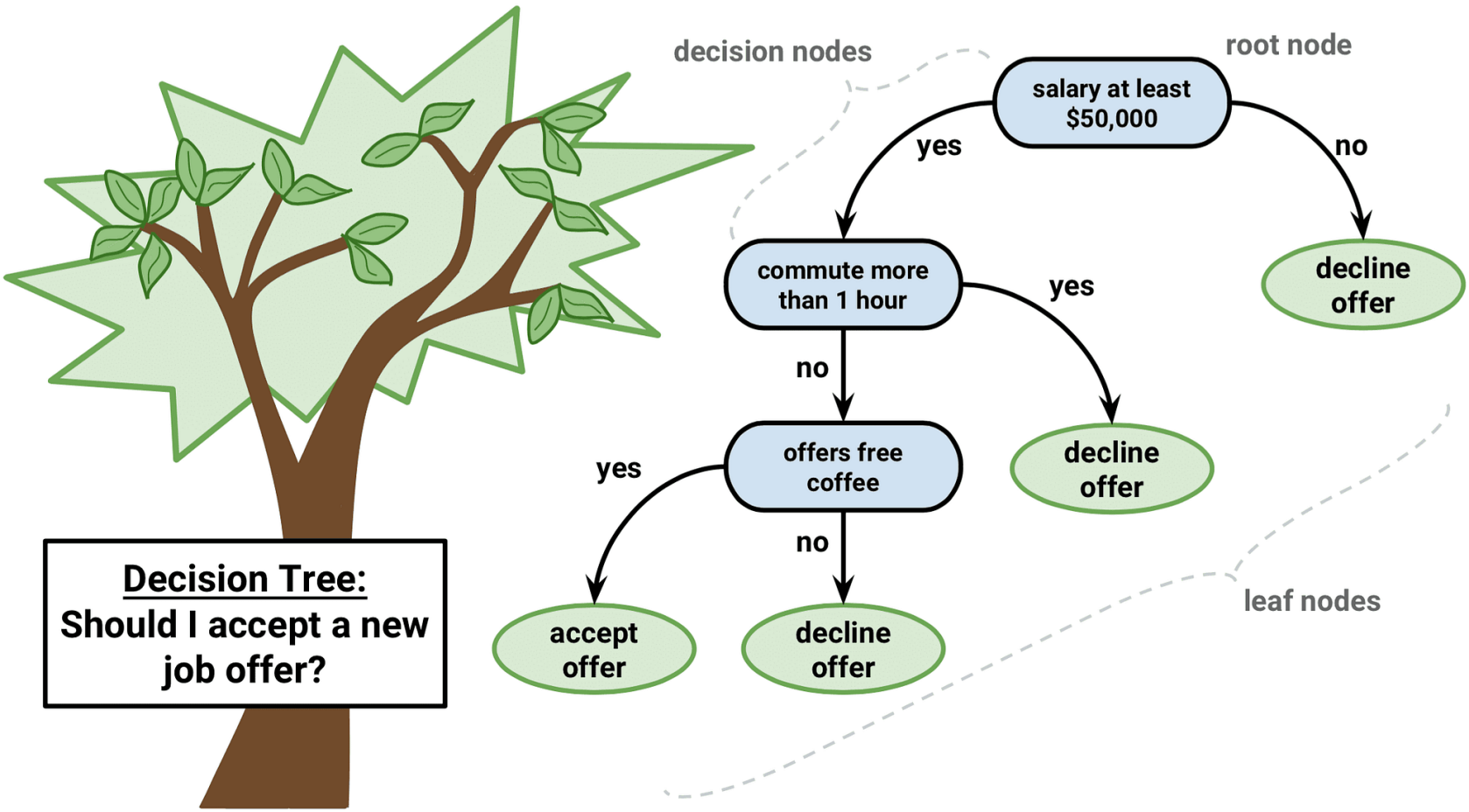
They allow efficient representation of hierarchical relationships and form the basis of:

- Search and decision structures (e.g. Binary Search Trees)
- Hierarchical data models (e.g. XML, file systems)
- Optimization algorithms and parsing

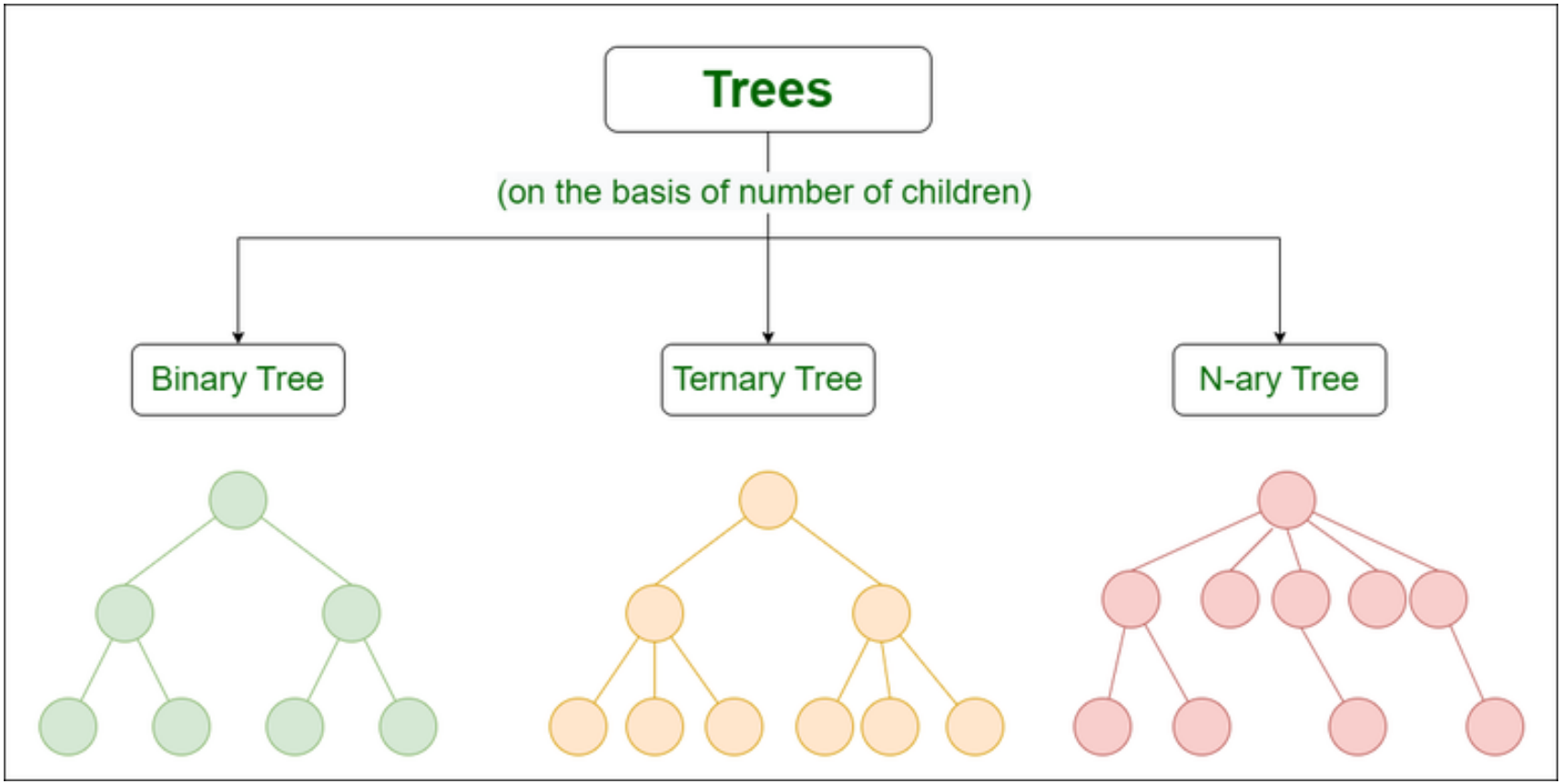
Tree (cont.)


Tree Data Structure
Root
Key
Edge
A
Parent
B
C
Subtree
Child
D
E
G
F
Siblings
H
I
J
K
L
M
N
O
P
Leaf Nodes
Height of the tree
Level 0
Level 1
Level 2
Level3
Level 4

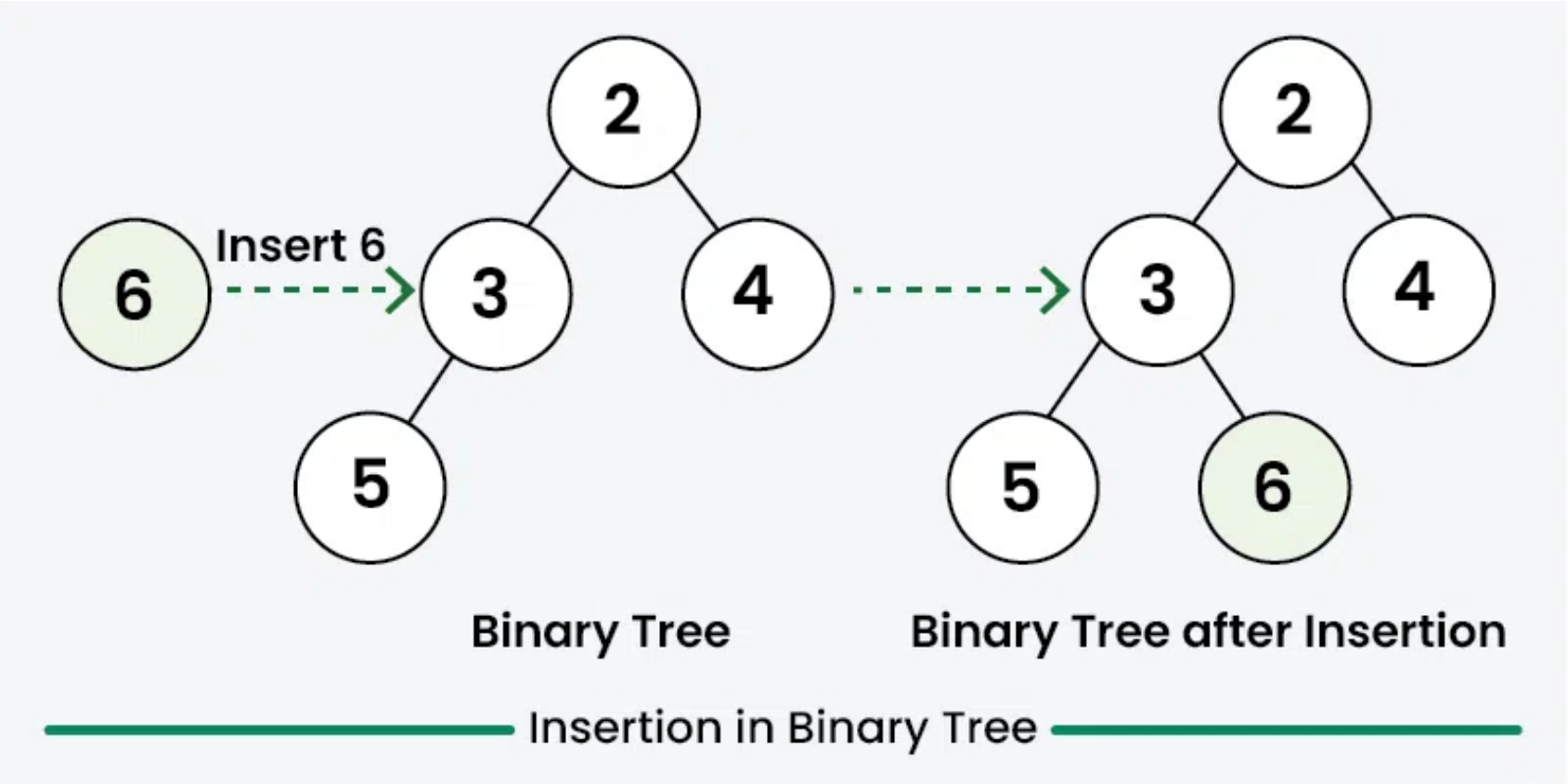
Tree application



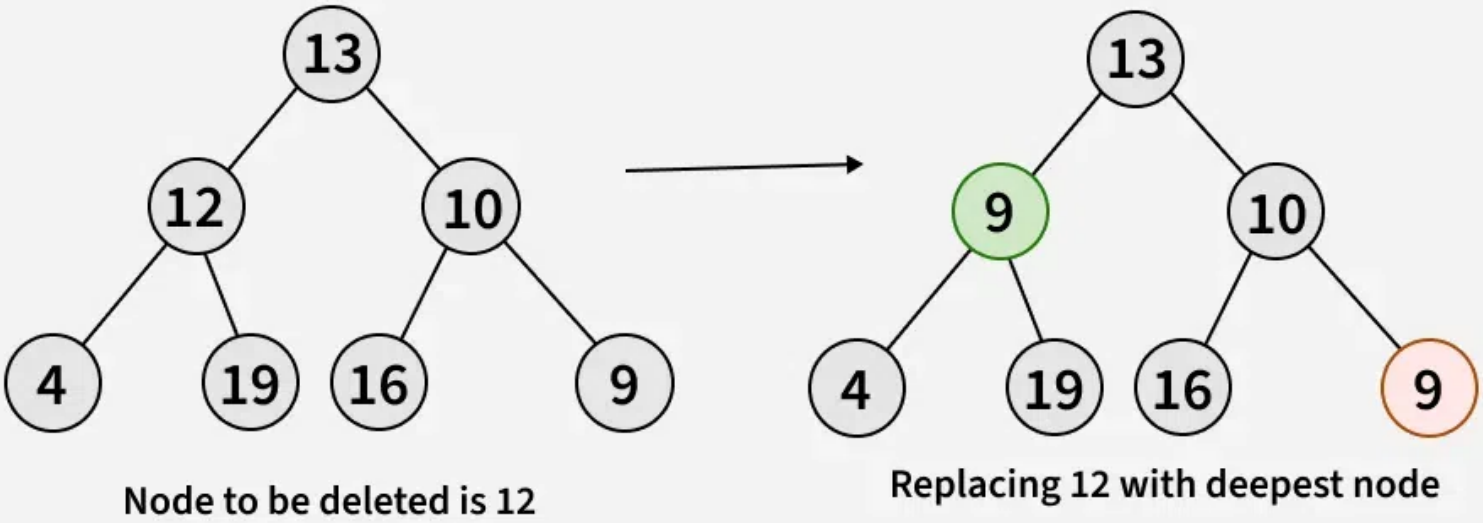
N-ary Tree, Ternary Tree, Binary Tree



Insertion in a binary tree



Deletion in a binary tree



Questions

Are the nodes sorted?

Questions

Are the nodes sorted? How can I look for a specific key in a tree?

Binary Search Trees (BST)

What is a Binary Search Tree?

A **Binary Search Tree (BST)** is a special type of **binary tree** where each node satisfies:

$$\text{left subtree values} < \text{node value} < \text{right subtree values}$$

- Each node has at most **two children**: a **left child** and a **right child**.
- The structure maintains a **sorted order**, enabling efficient search.
- Common operations: **insertion**, **search**, and **deletion**.

Binary Search Trees (BST)

What is a Binary Search Tree?

A **Binary Search Tree (BST)** is a special type of **binary tree** where each node satisfies:

$$\text{left subtree values} < \text{node value} < \text{right subtree values}$$

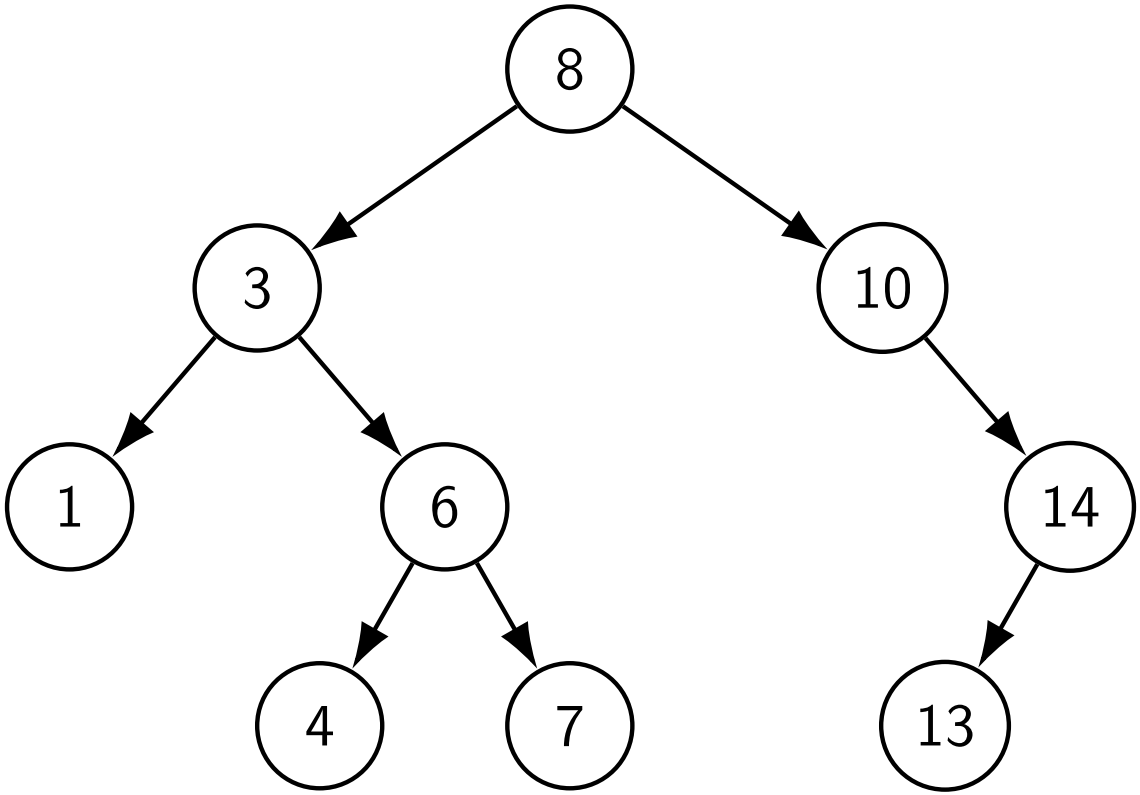
- Each node has at most **two children**: a **left child** and a **right child**.
- The structure maintains a **sorted order**, enabling efficient search.
- Common operations: **insertion**, **search**, and **deletion**.

Why are BSTs useful?

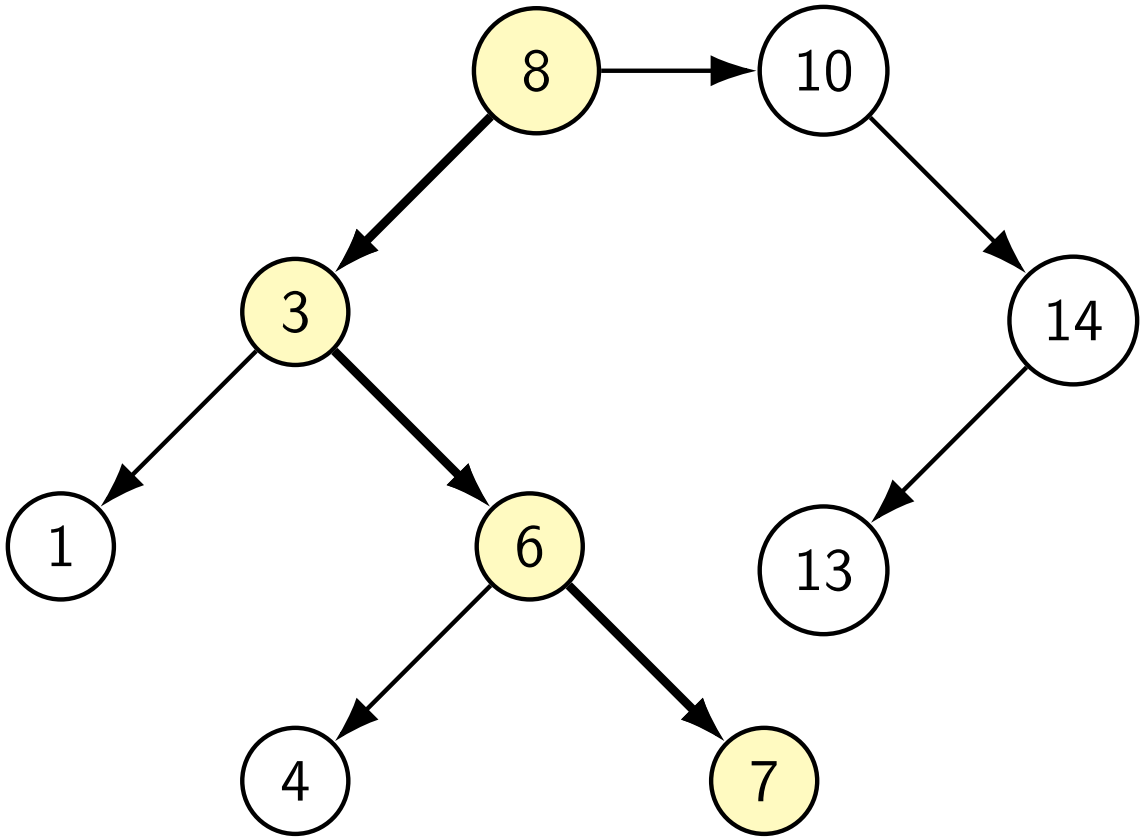
They allow:

- Searching in $O(\log n)$ time (on average)
- Maintaining dynamic, sorted data
- Forming the basis for balanced trees

BST — Example

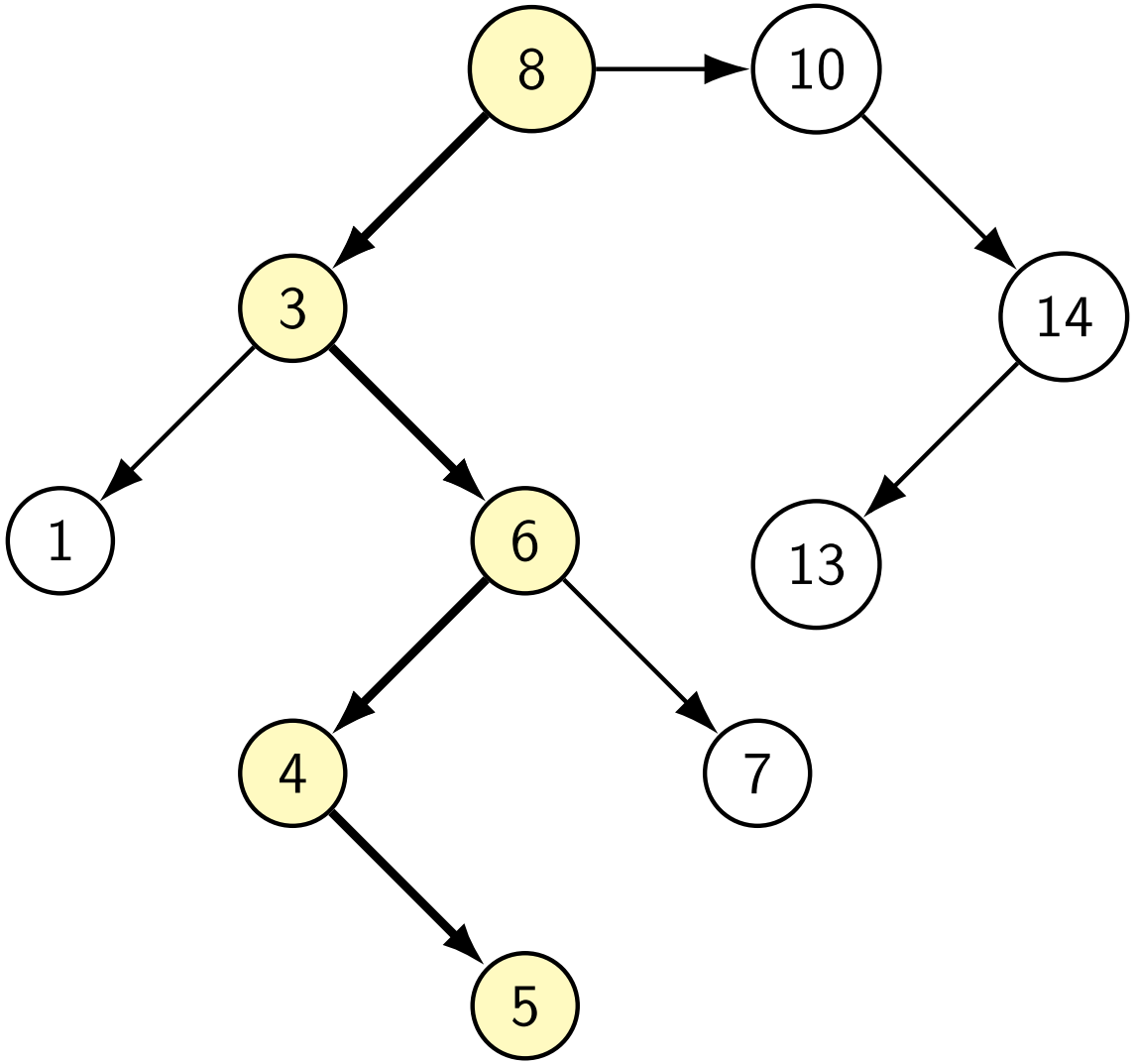


BST — Search for 7



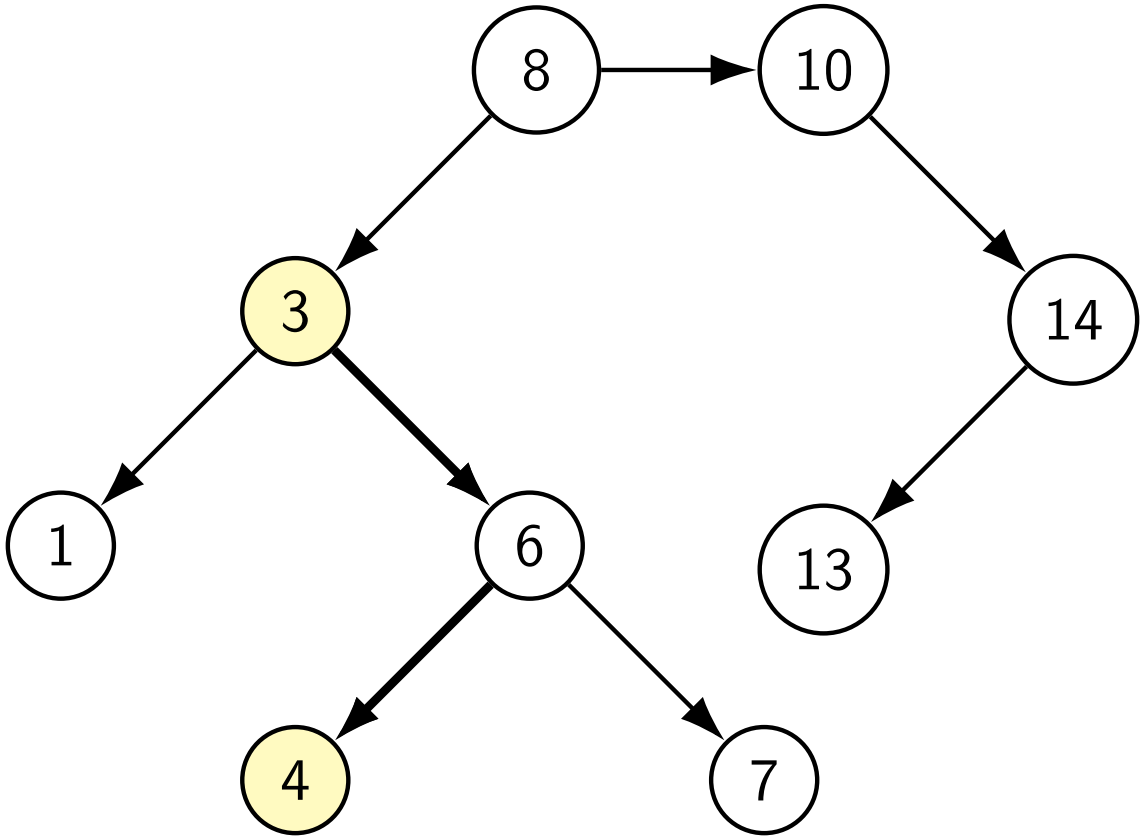
Path compared: $8 \rightarrow 3 \rightarrow 6 \rightarrow 7$ (found).

BST — Insertion of 5



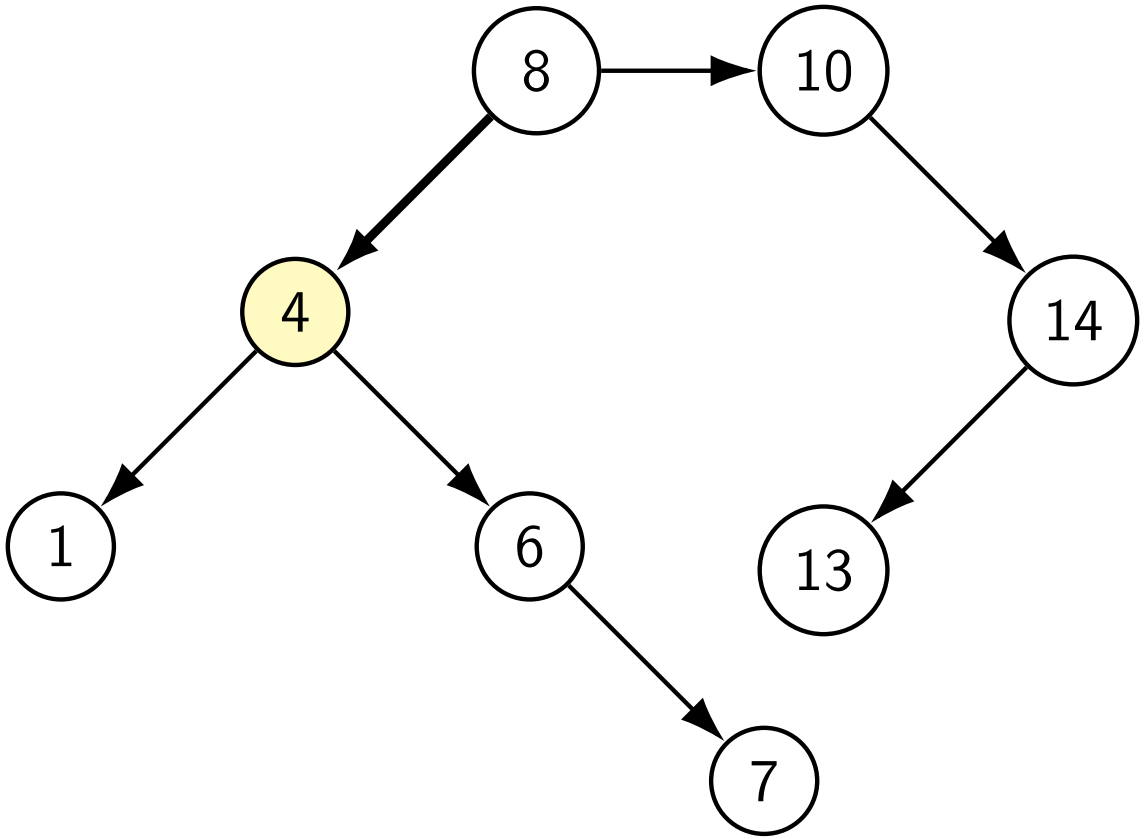
Insert path: $8 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow _(\text{right}) \Rightarrow 5$.

BST — Deletion of 3 (two children)



Step 1: Node 3 has two children. Inorder successor is 4.

BST — Deletion of 3 (two children)



Step 2: Copy 4 into node, then delete the original 4 (simple case).

BST in Python

BST in Python

```
class Node:
  def __init__(self, value):
    self.value = value
    self.left = None
    self.right = None

class BinarySearchTree:
  def __init__(self):
    self.root = None

  def insert(self, value):
    if not self.root: self.root = Node(value)
    else: self._insert_recursive(self.root, value)

  def _insert_recursive(self, node, value):
    if value < node.value:
      if not node.left: node.left = Node(value)
      else: self._insert_recursive(node.left, value)
    else:
      if not node.right: node.right = Node(value)
      else: self._insert_recursive(node.right, value)
```

BST through the OOP lens

- Two classes cooperate: `Node` objects hold the data and **two** references (`left`, `right`); the `BinarySearchTree` object holds only `root`

BST through the OOP lens

- Two classes cooperate: `Node` objects hold the data and **two** references (`left`, `right`); the `BinarySearchTree` object holds only `root`
- The **interface** is tiny: `insert(value)`. The user never creates a `Node` by hand

BST through the OOP lens

- Two classes cooperate: `Node` objects hold the data and **two** references (`left`, `right`); the `BinarySearchTree` object holds only `root`
- The **interface** is tiny: `insert(value)`. The user never creates a `Node` by hand
- `_insert_recursive` is a **helper** method: the leading underscore is a Python convention for "internal, not part of the interface"

BST through the OOP lens

- Two classes cooperate: `Node` objects hold the data and **two** references (`left`, `right`); the `BinarySearchTree` object holds only `root`
- The **interface** is tiny: `insert(value)`. The user never creates a `Node` by hand
- `_insert_recursive` is a **helper** method: the leading underscore is a Python convention for "internal, not part of the interface"
- The recursion walks from object to object by following the references, exactly as in the pictures

BST Insertions Can Produce an Unbalanced Tree

Claim. A Binary Search Tree (BST) built using naive insertions can become *highly unbalanced*.

Idea of the proof. Consider inserting the following sequence of keys:

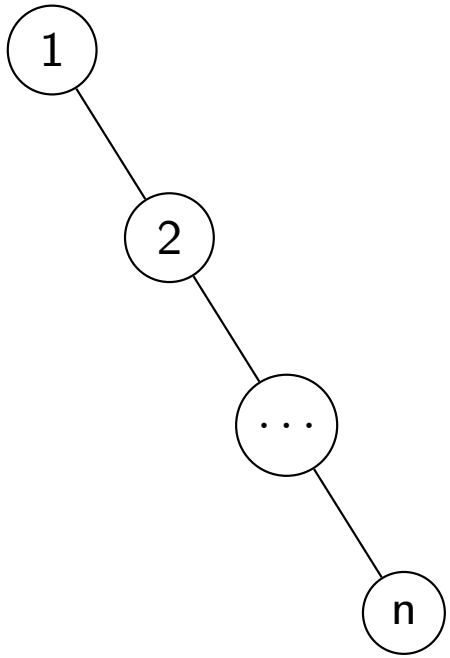
$$1, 2, 3, \ldots, n$$

- Insert 1: becomes the root.
- Insert 2: since $2 > 1$, it becomes the right child of 1.
- Insert 3: since $3 > 1$ and $3 > 2$, it goes even further right.
- The same happens for all successive elements.

Key observation: When keys arrive in sorted order, at every step the new node is placed as the *right child of the deepest node*, creating a chain.

Example of Degenerate BST

Resulting structure:



Height of the resulting tree:

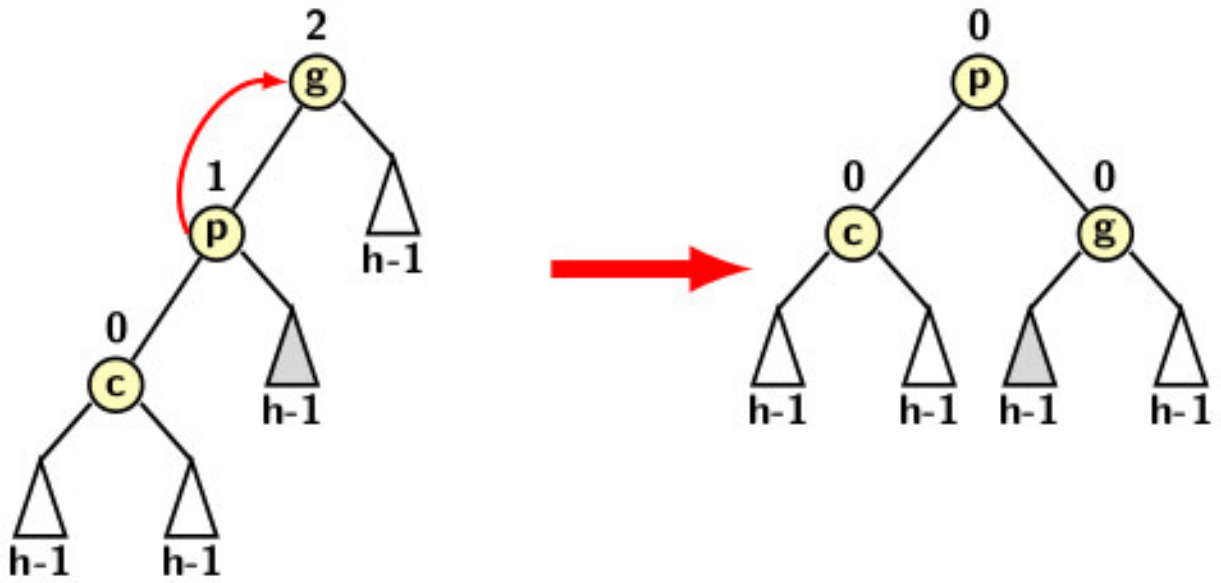
$$h = n - 1$$

Search time in this BST:

$O(n)$ (same as a linked list)

Conclusion. A naive BST provides no guarantee of balance: the insertion order alone can force the tree to degenerate.
Balanced trees (AVL, Red-Black, etc.) avoid this problem by performing **rotations**.

Rotation



There are different type of **Balanced BST** that changes according to the way they implement this rotation in insertion and/or deletion (AVL, Red-Black, etc.).

OOP in action: what we built today

Structure (classes)	Attributes (state)	Interface (methods)
Linked list `Node`, `LinkedList`	`value`, `next` `head`	`push_front`, `insert_after`, `delete_after`
Stack `Stack` / `LinkedStack`	`items` / `top`	`push`, `pop`, `peek`, `is_empty`
Queue `Node`, `Queue`	`front`, `back`	`enqueue`, `dequeue`, `peek`, `is_empty`
BST `Node`, `BinarySearchTree`	`value`, `left`, `right` `root`	`insert`

Key takeaway: a data structure is a class with a small, clear **interface**; the **implementation** behind it can change without changing the code that uses it.

The OOP vocabulary we used

Term	Meaning	Example
Class	template for creating objects	`class Stack:`
Object	a concrete instance with its own data	`my_stack = Stack()`
Constructor	method that sets up a new object	`def __init__(self): ...`
Attribute	data stored inside an object	`self.items`, `node.next`
Method	function that works on an object	`my_stack.push(10)`
`self`	the object a method is called on	`self.items.append(item)`